

Proyecto Individual: Desarrollo de una aplicación para la generación de gráficos y texto

Estudiante: Jose Andrés Arroyo
Meoño,
Carné: 2020092059
Instituto Tecnológico de Costa Rica
Arquitectura de Computadores 1
CE4301
Escuela de Ingeniería en
Computadores
Profesores: Luis Alberto Chavarría,
Jeferson González Gómez
I Semestre 2025

I. REQUERIMIENTOS

El objetivo principal de este trabajo es el diseño de un programa utilizando lenguaje ensamblador que permite aplicar el algoritmo de interpolación bilineal a una imagen. Esta debe tener una dimensión de al menos 390x390 píxeles, de tal manera que pueda ser dividida en 16 cuadrantes con los que se trabaja para obtener el acercamiento requerido.

Asimismo, el procesamiento de la imagen se realiza utilizando únicamente con el lenguaje ensamblador, del cual se eligió x86 (32 bit) de las opciones x86, ARM y RISC-V. Esta elección permite utilizar llamadas al sistema de manera eficiente para las operaciones de lectura y escritura de archivos, esenciales para el procesamiento de la imagen.

Además, dado que el proyecto trabaja con imágenes en escala de grises de 8 bits (valores entre 0 y 255), no se requiere el uso de un lenguaje ensamblador de 64 bits, ya que un tamaño de palabra de 32 bits es suficiente para manejar estos valores y simplificar el programa.

Aparte de la aplicación del algoritmo, Python permite a través de las librerías Pygame y Pillow, una visualización rápida e interactiva con la imagen inicial, que se encarga de procesar la solicitud del usuario y dar los valores iniciales al programa de ensamblador a través de un archivo personalizado .img, además de procesar el resultado y mostrarlo en pantalla.

II. SOLUCIÓN

Las soluciones tienen la forma general:

Python → .img → Ensamblador → .img → Python

Donde podemos dividir el proceso en dos; primero, la comunicación inicial donde Python envía los datos al ensamblador; y segundo, la devolución de resultados donde el ensamblador retorna la información procesada.

Por lo tanto, para encontrar una solución podemos detallar diferentes maneras en las que deben suceder ambas partes del proceso y elegir entre ellas la manera óptima con la que se debe construir el programa.

A. Comunicación Inicial

1) Utilizando buffers

La primera solución planteada es le da escribir en el archivo .img el cuadrante de la imagen de una forma matricial, de tal manera que cada columna este separada por un espacio y cada fila por el carácter de nueva línea \n. De esta manera, en ensamblador de podría tener dos buffers para leer un par de líneas a la vez y formar los grupos de cuatro píxeles requeridos para el cálculo de la interpolación.

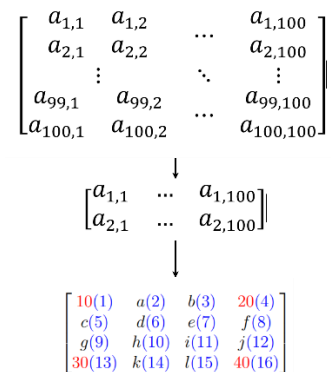


Fig 1. Diagrama de escritura de forma matricial y lectura de buffers en la comunicación inicial (tomando en cuenta una imagen 400x400 píxeles y cuadrantes 100x100)

Como se muestra en la figura, esta opción requiere dos buffers de al menos 400 bytes para albergar 100 números de 1 a 3 caracteres, 99 espacios que los separa y un carácter de nueva línea. Sería necesario un algoritmo que pueda leer 2 números de cada línea para realizar la interpolación.

2) Leyendo un bit a la vez

En este caso, en vez de escribir la imagen en forma matricial en el archivo .img, Python se encargaría de escribir la imagen en los grupos de cuatro píxeles necesitados, de tal manera que en ensamblador solo sea necesario procesar cuatro líneas para hacer el cálculo de la interpolación.

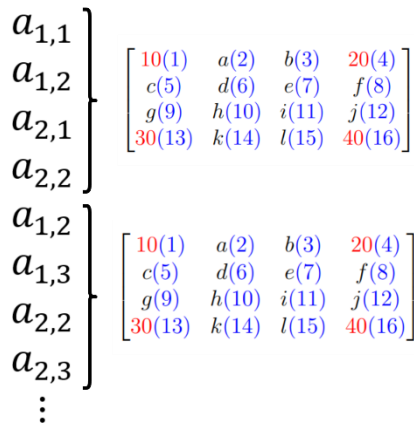


Fig 2. Diagrama de escritura un numero a la vez y lectura un bit a la vez en la comunicación inicial (tomando en cuenta una imagen 400x400 pixeles y cuadrantes 100x100)

En este algoritmo, cada valor estaría dividido por un carácter de nueva línea y, como se ve en el diagrama, al Python formar los grupos necesarios para la interpolación, en el archivo .img inicial la cantidad de valores pasaría de 10000 a 39204, ya que hay valores que se repiten.

B. Devolución de resultados

Nótese que, al calcular los valores de interpolación, se deben tomar matrices de 2x2 pixeles que resultan en matrices de 4x4 pixeles. Siempre que se encuentran dos matrices adyacentes, la arista que las une puede ser calculada tanto por la interpolación de una matriz como de la otra.

Esto hecho nos presenta con dos opciones para hacer la interpolación, tomar en cuenta los pixeles ya definidos o volverlos a calcular. Además, es importante definir la manera en la que el programa ensamblador escribe los valores en el archivo .img.

Por un lado, se pueden intentar guardar los nuevos valores dentro del archivo inicial y, por otro lado, se puede crear un nuevo archivo .img que guarde los nuevos valores, esto con el propósito de reducir el nivel de complejidad del algoritmo en lenguaje ensamblador encargado de escribir los resultados.

C. Comparación

En la comunicación inicial, por un lado, tenemos que al utilizar buffers aumentamos tanto la complejidad como el uso de memoria del programa ensamblador, pero reducimos la cantidad de datos necesaria en el archivo creado por Python.

Por otro lado, al leer cada byte del archivo disminuimos la complejidad y uso de memoria del algoritmo, ya que reducimos el proceso a la repetición de una operación básica. Aun así, este proceso aumenta la cantidad de datos en el archivo de Python.

En la devolución de los resultados, implementar un sistema para evitar recalculer los pixeles que comparten diferentes matrices aumentaría la complejidad del algoritmo, pero nos permitiría hacer que el algoritmo sea más rápido al eliminar operaciones redundantes. Por el contrario, optar por calcular todos los pixeles de forma independiente, sin verificar si fueron previamente procesados, simplificaría la

estructura del algoritmo, pero aumentaría la cantidad de operación realizadas.

Por último, escribir los resultados de los cálculos en un nuevo archivo solo presenta una mayor facilidad sobre escribirlos en el archivo original.

III. PROPUESTA FINAL

El sistema propuesto comienza con la generación de un archivo .img inicial donde los valores se almacenan línea por línea, con el objetivo deliberado de facilitar y hacer más eficiente la lectura en el programa ensamblador. Se debe tomar en cuenta que esta decisión incrementa ligeramente el trabajo de preparación en Python y el tamaño del archivo inicial.

En cuanto al algoritmo de interpolación, se optó por una implementación que calcula todos los valores de forma independiente, sin reutilizar los resultados previamente obtenidos para pixeles adyacentes, lo cual presenta un punto de mejora del programa, ya que, aunque es un enfoque más sencillo, es menos eficiente. Además, sus resultados se guardaron en un archivo aparte para facilitar su escritura, lo que aumenta un poco la complejidad de la lectura para visualizar la imagen en Python.

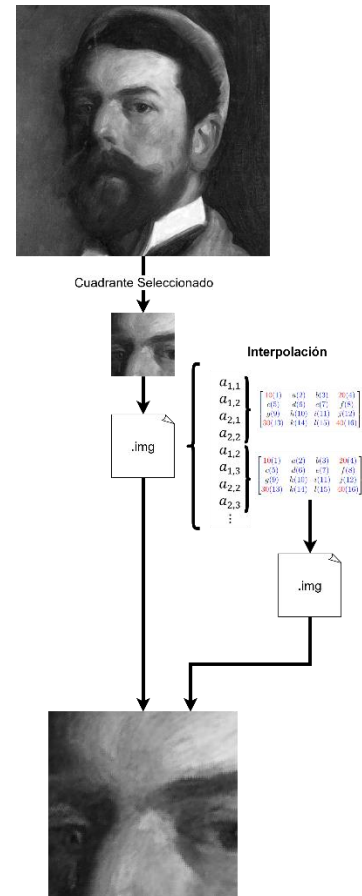


Fig 3. Diagrama de programa completo